UNIVERSIDAD NACIONAL DE COLOMBIA

FACULTAD DE INGENIERÍA

DEPARTAMENTO DE INGENIERÍA DE SISTEMAS E INDUSTRIAL

LENGUAJES DE PROGRAMACIÓN

TAREA # 29 — LIBRERÍA BÁSICA DE ESTRUCTURAS DE DATOS (LISTAS Y MATRICES) PARA LA MÁQUINA DE LA TAREA # 10 (ENIGMA-64)

AUTORES:

Juan Sebastian Umaña Camacho

Tomas Felipe Garzón Gómez

Michael Stiven Betancourt Gelves

Juan Luis Vergara Novoa

Maicol Sebastian Olarte Ramirez

Deibyd Santiago Barragán Gaitán

Alejandro Argüello Muñoz

Docente: Jorge Eduardo Ortiz Triviño

Bogotá D.C., Colombia — Noviembre de 2026

CONTENIDO

Sección	Título	
—	Enunciado y equipo de trabajo	2
1	Marco teórico	3
2	Descripción y justificación del problema	4
3	Diseño de la solución (bloques, mapa de memoria, clases y flujo)	5
4	La librería: módulos, API y convención de llamada	7
5	Los tres problemas computacionales	8
6	Manual técnico	10
7	Manual de usuario (línea de comandos y GUI Noctua)	11
8	Experimentación y análisis de resultados	12
9	Limitaciones conocidas	14
10	Conclusiones	14
—	Referencias	15
A	Anexo: inventario del código fuente	15

Correspondencia con los entregables de una tarea automatizada. La tarea se entrega en un único archivo; la tabla indica dónde está cada entregable mínimo exigido:

Entregable exigido	Ubicación en este documento
a. Marco teórico	Sección 1
b. Descripción y justificación del problema	Sección 2
c. Diseño de la aplicación (diagramas de bloques, de flujo y UML)	Secciones 3 y 5 (Diagramas 1 a 5)
d. Código fuente completo y versiones de las librerías	Sección 6.2 (versiones) y Anexo A; el código acompaña la entrega en spl/include/, spl/estructuras/, spl/ejemplos/ y enigma64/ui/
e. Manual de usuario y manual técnico	Secciones 6 y 7
f. Experimentación y análisis de al menos 3 escenarios	Sección 8 (merge sort, producto punto / búsqueda binaria, matrices)
Información de todos los autores	Portada y tabla de roles del equipo

ENUNCIADO

1. Análisis, diseño e implementación de una pequeña librería de gestión de estructuras de datos básicas: listas (vectores) y matrices. La librería debe ser importada o incluida por el programador en sus aplicaciones para que sea añadida y gestionada por el preprocesador del sistema de procesamiento del lenguaje.

2. Pruebe su funcionamiento con el algoritmo de las mezclas utilizado para ordenar datos almacenados en una lista. Seleccione otros dos problemas computacionales que empleen vectores y matrices.

Datos de la tarea: inicio el 30 de octubre de 2026, entrega el 13 de noviembre de 2026; categoría automatizada, obligatoria, clase 15; grupo de Taller-Proyecto de 7 estudiantes.

Equipo de trabajo y roles

El grupo es el mismo de las Tareas # 09, # 10, # 27 y del Taller # 01. Cada integrante conservó el subsistema de Enigma-64 del que era responsable y construyó la parte de la librería que lo toca:

Int.	Nombre	Módulo	Responsabilidad en la Tarea 29
1	Juan Sebastian Umaña Camacho	layout_datos.inc, preprocesador	#include "estructuras.inc", macros parametrizadas VEC_DIR / MAT_DIR y mapa de memoria de datos alineado a 8 bytes.
2	Tomas Felipe Garzón Gómez	vec.asm	Primitivas 1D: VEC_ADDR, VEC_GET, VEC_SET, VEC_SWAP, VEC_COPY, VEC_LEN; convención R1–R4 / R5.
3	Michael Stiven Betancourt Gelves	mat.asm	Primitivas 2D: MAT_ADDR, MAT_GET, MAT_SET, MAT_CLEAR y marcos de pila (PUSH/POP, ENTER/LEAVE, CALL/RET).
4	Juan Luis Vergara Novoa	estructuras.inc, cabeceras.asm, spl.estructuras	Archivo maestro, descriptores y cabeceras, normalización a 64 bits y puente Preprocesador → Ensamblador → .e64 → CargadorEnigma.
5	Maicol Sebastian Olarte Ramirez	enigma64/ui/	Pestaña Estructuras de la GUI Noctua: VisorVector, VisorMatriz y AdaptadorEstructuras; integración de los tres problemas.
6	Deibyd Santiago Barragán Gaitán	merge_sort.asm	Algoritmo de las mezclas con vector auxiliar y salida por la pantalla MMIO (0xFF001010).
7	Alejandro Argüello Muñoz	producto_punto, busqueda_binaria, multiplicacion_matrices, tests	Problemas 2 y 3 y la suite automatizada tests/test_libreria_estructuras.py.

1. MARCO TEÓRICO

1.1 Tipos abstractos de datos lineales: listas y matrices

Un *tipo abstracto de datos* (TAD) separa la especificación de un tipo (sus operaciones y propiedades) de su representación. La lista es la secuencia finita $V = (v_0, \ldots, v_{N-1})$ con las operaciones obtener, asignar, intercambiar, copiar y longitud; la matriz M de $F \times C$ es una función de los pares (i, j), $0 \le i < F$, $0 \le j < C$, en los valores. Cuando la representación es *contigua* y los elementos tienen tamaño fijo w, el acceso es de tiempo constante porque la dirección se calcula: $\mathrm{Dir}(v_i) = B + i \cdot w$ [1]. En Enigma-64 cada elemento es una palabra de 64 bits ($w = 8$), de modo que la multiplicación se reduce a un desplazamiento: **Dir(V[i]) = B + (i << 3)**.

Una matriz se aplana a un vector con un *orden de almacenamiento*. En orden por filas (row-major, el de C y Pascal) la fila i ocupa C palabras consecutivas que empiezan en $B + i \cdot C \cdot w$, de donde **Dir(M[i][j]) = B + ((i·C + j) << 3)**. El orden por filas tiene una consecuencia útil: cada fila es en sí misma un vector, así que las primitivas de listas operan sobre filas sin conversión. La fórmula necesita C, el *vector de información* o dope vector de la matriz; por eso la librería guarda una cabecera con las dimensiones junto a los datos.

1.2 Preprocesador, inclusión de archivos y macros

El preprocesador es la primera fase del sistema de procesamiento del lenguaje: transforma el texto antes del ensamblado. La directiva `#include` sustituye la línea por el contenido del archivo, lo que permite distribuir una librería como fuente y que el programador la «importe» con una sola línea; `#define` declara macros objeto y macros parametrizadas (VEC_DIR(base, i)), y `#ifdef / #ifndef / #pragma once` evitan la doble inclusión y permiten activar módulos opcionales (INCLUIR_MATRICES) [2]. El preprocesador del SPL de Enigma-64 se construyó con ply.lex en el Taller # 01: reconoce las directivas con un analizador léxico de estados y resuelve las inclusiones recursivas con detección de ciclos y búsqueda primero junto al archivo y luego en `spl/include/`.

1.3 Subrutinas, convención de llamada y marcos de pila

Una librería en ensamblador solo es reutilizable si todas sus rutinas respetan el mismo *contrato binario* (ABI): dónde van los argumentos, dónde el resultado y qué registros conserva la rutina llamada (callee-saved) frente a los que puede destruir (caller-saved). Las rutinas que llaman a otras guardan sus temporales con PUSH y abren un *registro de activación* con ENTER t (apila BP, BP = SP, reserva t bytes de variables locales); LEAVE y RET lo deshacen en orden inverso. La cadena de BP guardados (enlace dinámico) permite recorrer la pila y es lo que hace posible la recursión del algoritmo de las mezclas [3].

1.4 Los algoritmos de prueba

Ordenamiento por mezclas (merge sort). Algoritmo de divide y vencerás de von Neumann (1945): divide la lista en dos mitades, las ordena recursivamente y las *mezcla* en tiempo lineal con un vector auxiliar. Su costo cumple $T(N) = 2T(N/2) + \Theta(N)$, es decir $\Theta(N \log N)$ en todos los casos, y es estable [1, 4]. **Producto punto.** $a \cdot b = \Sigma\, a_k b_k$, $\Theta(N)$, recorrido secuencial de dos vectores. **Búsqueda binaria.** Sobre un vector ordenado compara con el elemento central y descarta la mitad: $\Theta(\log N)$ comparaciones. **Producto de matrices.** $C = A \cdot B$ con $c_{ij} = \Sigma_k\, a_{ik} b_{kj}$, $\Theta(F \cdot K \cdot C)$ con el algoritmo de tres bucles; y su caso particular matriz por vector, $y = A \cdot x$, $\Theta(F \cdot K)$. Juntos ejercitan todas las operaciones de la librería: acceso aleatorio, escritura, copia, recursión, y acceso 2D.

2. DESCRIPCIÓN Y JUSTIFICACIÓN DEL PROBLEMA

2.1 Descripción

Enigma-64 (Tareas # 09 y # 10) es una máquina Von Neumann multiciclo de 64 bits con 8 registros (R0 = 0, R6 = SP, R7 = BP), inmediatos de 16 bits con signo, memoria big-endian de 4 GiB con palabras alineadas a 8 bytes (un acceso desalineado produce STATUS_MISALIGNED) y la zona de sistema por debajo de 0x00200000 protegida. Su ensamblador no tiene tipos ni directivas de datos: un programador que necesite una lista o una matriz debe calcular direcciones a mano, decidir dónde guarda la longitud y repetir ese código en cada programa. El problema es ofrecerle una librería que resuelva eso una sola vez y que se incorpore a cualquier programa con `#include "estructuras.inc"`, gestionada por el preprocesador del SPL.

Los requisitos que se derivan del enunciado y de la máquina son: (R1) inclusión por el preprocesador, sin pasos manuales; (R2) acceso en O(1) a V[i] y M[i][j]; (R3) metadatos (longitud, filas, columnas) en memoria, legibles en tiempo de ejecución; (R4) alineación a 8 bytes y respeto del mapa de memoria de la Tarea # 09; (R5) una convención de llamada única que permita anidar y recursar; (R6) modularidad: cada módulo se puede incluir y probar solo; (R7) demostración con el merge sort y otros dos problemas sobre vectores y matrices.

2.2 Justificación

Había tres caminos. (i) **Solo macros**: VEC_DIR y MAT_DIR resuelven la dirección en el texto, pero el preprocesador de una línea por macro no puede emitir secuencias de instrucciones, y los índices que viven en registros no se pueden calcular en tiempo de preprocesamiento. (ii) **Extender el ensamblador** con instrucciones o directivas de estructuras: rompe la compatibilidad con la máquina de la Tarea # 10, que no debe cambiar. (iii) **Librería de subrutinas en ensamblador incluida por el preprocesador**, el camino elegido: las macros quedan para constantes y direcciones conocidas al escribir el programa y las subrutinas para todo lo dinámico. Ventajas: el código es ensamblador de Enigma-64 estándar que corre a velocidad nativa; cada archivo es un módulo con guardas de inclusión; la convención R1–R4 / R5 hace que las rutinas se compongan; y el lado Python (spl.estructuras) reutiliza el mismo formato de descriptores para precargar datos, leer resultados, probar y visualizar en la GUI.

3. DISEÑO DE LA SOLUCIÓN

3.1 Arquitectura general (diagrama de bloques)

El Diagrama 1 muestra cómo se organiza la solución en capas independientes: a la izquierda los clientes (el programa del usuario, la GUI y las pruebas), en el centro la librería en ensamblador (recuadro punteado) y el paquete Python que conecta ambos mundos, y a la derecha, en gris, el sistema de procesamiento del lenguaje y la máquina de la Tarea # 10.

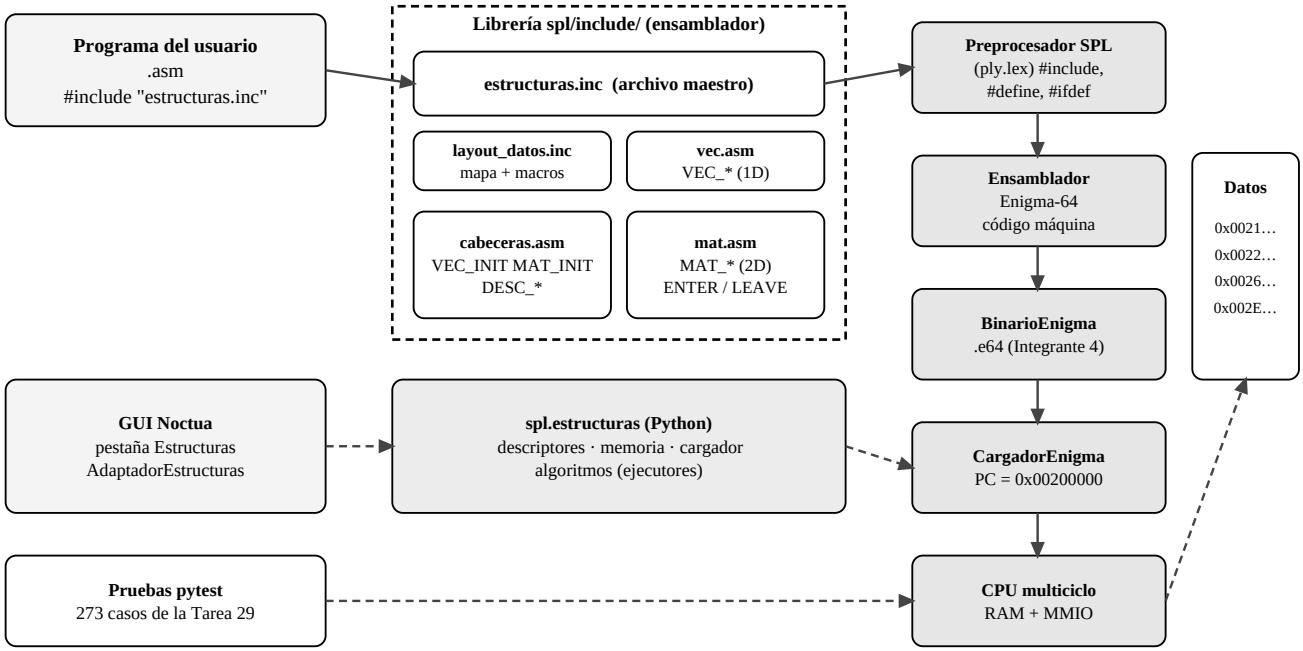


Diagrama 1: Arquitectura de la librería y su integración con el SPL y con Enigma-64. Flecha continua: inclusión o flujo de datos; discontinua: uso.

3.2 Diseño modular

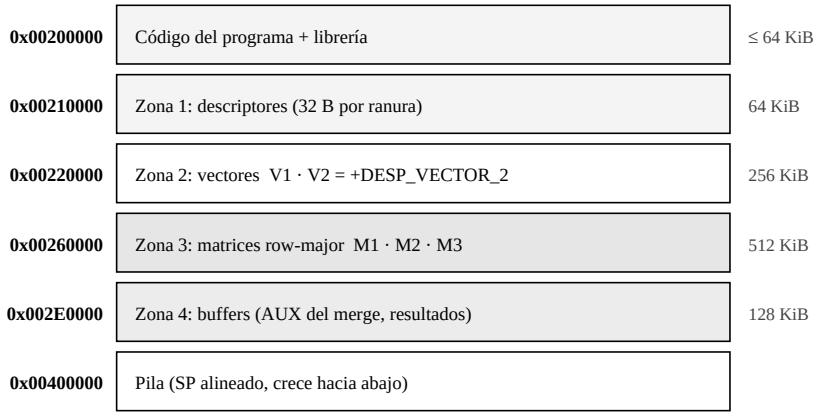
La modularidad es la decisión central del diseño. Cada archivo de `spl/include/` es un módulo con una sola responsabilidad, una guarda `#pragma once` + `#ifndef` y un `JMP` inicial que salta su cuerpo, de modo que puede incluirse al inicio o al final del programa sin ejecutarse por caída. El programador elige el grano: `#include "vec.asm"` solo trae listas; `#include "estructuras.inc"` trae listas y cabeceras; y `#define INCLUIR_MATRICES` antes de ese include añade `mat.asm`. Las dependencias forman un árbol sin ciclos: mat.asm → cabeceras.asm; estructuras.inc → layout_datos.inc, vec.asm, cabeceras.asm [, mat.asm].

Módulo	Responsabilidad	Depende de
layout_datos.inc	Mapa de las 4 zonas de datos, DESP_VECTOR_2 / DESP_MATRIZ_2 / DESP_MATRIZ_3 y macros VEC_DIR, MAT_DIR	—
vec.asm	Primitivas 1D (hojas, sin pila salvo VEC_SWAP / VEC_COPY)	—
cabeceras.asm	Cabecera inline y tabla de descriptores (tipo, base, filas, cols)	—
mat.asm	Primitivas 2D (no hoja, con marcos ENTER/LEAVE)	cabeceras.asm
estructuras.inc	Punto único de inclusión, constantes del ABI y DESC_DIR(slot)	los anteriores
spl/estructuras/*.py	Descriptores, precarga/lectura en RAM, compilación a .e64, ejecutores	preprocesador, ensamblador, cargador

Módulo	Responsabilidad	Depende de
enigma64/ui/…/estructuras.py	Adaptador y pestaña de la GUI	spl.estructuras

Los algoritmos de prueba (MERGE_SORT, VEC_DOT, VEC_BIN_SEARCH, MAT_MUL, MAT_VEC_MUL) viven en sus programas de `spl/ejemplos/` y no en la librería: son *aplicaciones* de la librería, como pide el enunciado, y la librería se mantiene pequeña y básica. Durante la integración se eliminaron los desplazamientos codificados a mano (256, 512, 1024 bytes) de los programas 2 y 3 y se reemplazaron por las macros DESP_* del mapa de memoria, que ahora es la única fuente de verdad para Python, ensamblador y GUI (Sección 8.6).

3.3 Mapa de memoria y representación



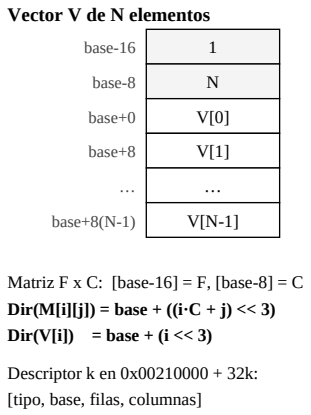


Diagrama 2: Zonas de datos de layout_datos.inc (izquierda) y representación de un vector con su cabecera inline (derecha).

Cada estructura tiene un **modelo dual de metadatos**: una cabecera *inline* de dos palabras justo antes de los datos (lo que leen VEC_LEN, MAT_ROWS y MAT_COLS en O(1)) y, opcionalmente, un *descriptor* de 32 bytes en la tabla de la Zona 1 (lo que permite a la GUI y a las pruebas descubrir qué estructuras construyó un programa). Todas las bases son múltiplos de 8; la Zona 1 empieza en 0x00210000 para no pisar los programas demo de la Tarea # 10 (0x00201000–0x00205020).

3.4 Diagrama de clases del lado Python (UML)

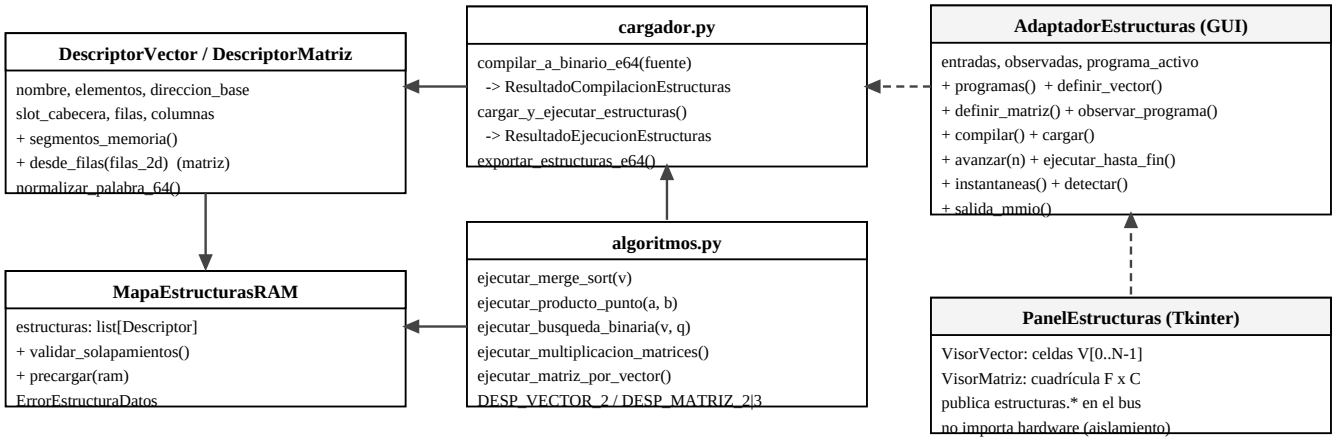


Diagrama 3: Clases y módulos de spl.estructuras (izquierda y centro) y de la GUI (derecha). Flecha continua: uso directo; discontinua: el adaptador usa el puente.

`MapaEstructurasRAM` valida antes de escribir que ningún segmento (cabecera inline, datos, descriptor) se solape con otro ni salga de su zona, y lanza `ErrorEstructuraDatos` con las direcciones exactas del conflicto. `cargar_y_ejecutar_estructuras` encadena preprocesador, ensamblador, empaquetado .e64, CargadorEnigma, precarga de las estructuras y ejecución en la CPU.

4. LA LIBRERÍA: MÓDULOS, API Y CONVENCIÓN DE LLAMADA

4.1 Convención de llamada (ABI)

Elemento	Regla
Argumentos	R1, R2, R3, R4 en ese orden (del 5.º en adelante, por pila)
Retorno	R5 (alias RV)
Preservados (callee-saved)	R1–R4, R6 = SP, R7 = BP
Destruidos (caller-saved)	R5 y banderas Z, N, C, V
Pila	Crece hacia direcciones menores; SP alineado a 8 (0x00400000) antes de la primera llamada; cada rutina deja SP como lo encontró
Límites	Las primitivas no validan índices (como en C): 0 ≤ i < N es responsabilidad del llamador; los datos son 64 bits crudos, válidos para enteros y double IEEE-754

4.2 Interfaz de programación

Rutina	Entradas	Salida / efecto	Módulo
VEC_ADDR	R1 = base, R2 = i	R5 = base + (i << 3)	vec.asm
VEC_GET / VEC_SET	R1, R2 = i (, R3 = valor)	R5 = V[i] / V[i] = R3	vec.asm
VEC_SWAP	R1, R2 = i, R3 = j	intercambia V[i] y V[j] (correcto con i = j)	vec.asm
VEC_COPY	R1 = dest, R2 = orig, R3 = n	copia n palabras, semántica memmove	vec.asm
VEC_LEN	R1 = base	R5 = N (de [base – 8])	vec.asm
VEC_INIT / MAT_INIT	R1 = base, R2 = N | F (, R3 = C)	escribe la cabecera inline	cabeceras.asm
MAT_ROWS / MAT_COLS	R1 = base	R5 = F / C	cabeceras.asm

Rutina	Entradas	Salida / efecto	Módulo
DESC_VEC_INIT / DESC_MAT_INIT	R1 = ranura, R2 = base, R3 (, R4)	descriptor en 0x00210000 + 32·ranura y cabecera inline	cabeceras.asm
DESC_ADDR, DESC_GET_*	R1 = ranura	R5 = dirección / tipo / base / filas / cols	cabeceras.asm
MAT_ADDR	R1 = base, R2 = i, R3 = j	R5 = base + ((i·C + j) << 3)	mat.asm
MAT_GET / MAT_SET	R1, R2 = i, R3 = j (, R4 = valor)	R5 = M[i][j] / M[i][j] = R4	mat.asm
MAT_CLEAR	R1 = base	pone a 0 las F·C palabras; R5 = F·C	mat.asm

4.3 Ejemplo de uso mínimo

El siguiente programa completo crea V = [7, 3] y los intercambia; todo lo que no es `PRINCIPAL` lo aporta la librería:

```
#include "estructuras.inc"          ; el preprocesador trae layout, vec y cabeceras
PRINCIPAL:
        ADDI SP, R0, 0x0040
        SHL  SP, SP, 16             ; SP = 0x00400000 (alineado)
        ADDI R1, R0, 0x0022
        SHL  R1, R1, 16             ; R1 = BASE_VECTORES
        ADDI R2, R0, 2
        CALL VEC_INIT               ; cabecera: N = 2
        ADDI R2, R0, 0
        ADDI R3, R0, 7
        CALL VEC_SET                ; V[0] = 7
        ADDI R2, R0, 1
        ADDI R3, R0, 3
        CALL VEC_SET                ; V[1] = 3
        ADDI R2, R0, 0
        ADDI R3, R0, 1
        CALL VEC_SWAP               ; V = [3, 7]
        HLT
```

MAT_GET muestra la gestión de pila de una rutina no hoja: abre un marco con `ENTER 0`, llama a MAT_ADDR (segundo nivel), lee y deshace el marco. Durante esa llamada la pila contiene, de arriba abajo, el retorno a MAT_GET, el BP del llamador y el retorno al llamador; las pruebas del Integrante 3 inspeccionan esos marcos en la CPU real y ejecutan 300 niveles de recursión.

```
MAT_ADDR:   LOAD R5, [R1 - 8]       ; C de la cabecera
            MUL  R5, R2, R5         ; i * C
            ADD  R5, R5, R3         ; i * C + j
            SHL  R5, R5, 3          ; * 8 bytes
            ADD  R5, R1, R5         ; base + ...
            RET
MAT_GET:    ENTER 0                 ; marco: apila BP, BP = SP
            CALL MAT_ADDR           ; nivel 2
            LOAD R5, [R5]
            LEAVE
            RET
```

5. LOS TRES PROBLEMAS COMPUTACIONALES

Los cuatro programas siguen el mismo patrón: incluyen la librería, ponen SP en 0x00400000, comprueban si la cabecera de su primera estructura ya tiene datos (precargados desde Python o desde la GUI) y, si no, construyen unos datos de demostración; al terminar dejan los resultados en RAM, en R5 y, en el merge sort, en la pantalla MMIO. Así cada programa es autónomo y a la vez parametrizable.

5.1 Problema 1: algoritmo de las mezclas (merge_sort.asm, Integrante 6)

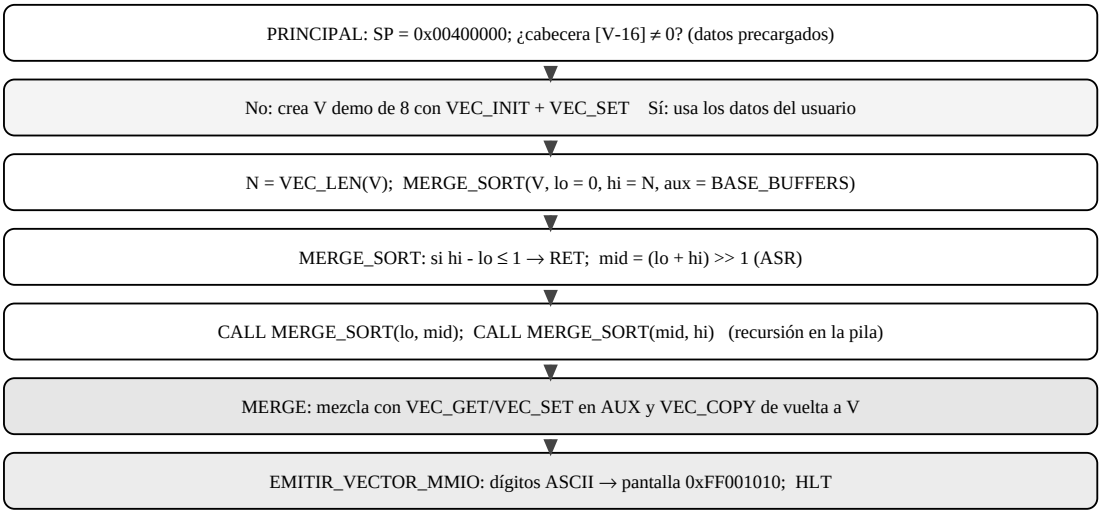


Diagrama 4: Flujo de merge_sort.asm. Cada caja usa rutinas de la librería: VEC_INIT, VEC_SET, VEC_LEN, VEC_GET, VEC_COPY.

MERGE_SORT(V, lo, hi, aux) ordena el intervalo semiabierto [lo, hi). El punto medio se calcula con ASR para no desbordar; R1 (base), R2 (lo), R4 (aux) se preservan por la ABI, de modo que las dos llamadas recursivas no necesitan restaurarlos. MERGE copia [lo, hi) a AUX con VEC_COPY y mezcla de vuelta sobre V eligiendo el menor con VEC_GET (≤ para que el orden sea estable). EMITIR_VECTOR_MMIO convierte cada entero con signo a dígitos ASCII y los escribe en el registro DATA de la pantalla (0xFF001010).

5.2 Problema 2: vectores — producto punto y búsqueda binaria (Integrante 7)

producto_punto.asm lee A y B (A en BASE_VECTORES, B en BASE_VECTORES + DESP_VECTOR_2), acumula A[k]·B[k] con VEC_GET y MUL en VEC_DOT y deja el resultado en el vector DOT de longitud 1 en BASE_BUFFERS y en R5. **busqueda_binaria.asm** recibe un vector ordenado V y un vector de consultas Q; para cada Q[k] VEC_BIN_SEARCH reduce [lo, hi] con mid = (lo + hi) >> 1 y deja el índice (o −1) en R[k] (BASE_BUFFERS). Los dos problemas cubren el recorrido secuencial Θ(N) y el acceso aleatorio Θ(log N) sobre listas.

5.3 Problema 3: matrices — C = A × B y y = A × x (Integrante 7)

multiplicacion_matrices.asm se incluye con `#define INCLUIR_MATRICES`. MAT_MUL lee las dimensiones con MAT_ROWS / MAT_COLS, crea C (FA × CB) con DESC_MAT_INIT en BASE_MATRICES + DESP_MATRIZ_3, la limpia con MAT_CLEAR y ejecuta el triple bucle con MAT_GET y MAT_SET; sus nueve variables locales viven en un marco `ENTER 72` ([BP – 8] … [BP – 72]). Si hay un vector x en BASE_VECTORES, MAT_VEC_MUL calcula también y = A·x mezclando primitivas 2D (MAT_GET) y 1D (VEC_GET, VEC_SET), lo que demuestra que ambos módulos se componen.

```
MM_BODY_K:  LOAD R1, [BP - 8]       ; base_A
            LOAD R2, [BP - 56]      ; i
            ADD  R3, R4, R0         ; k
            CALL MAT_GET            ; R5 = A[i][k]
            PUSH R5
            LOAD R1, [BP - 16]      ; base_B
            ADD  R2, R4, R0         ; k
            LOAD R3, [BP - 64]      ; j
            CALL MAT_GET            ; R5 = B[k][j]
            POP  R3
            MUL  R5, R3, R5         ; acc += A[i][k] * B[k][j]
```

6. MANUAL TÉCNICO

6.1 Estructura del paquete

Archivo	Contenido
spl/include/estructuras.inc	Archivo maestro: constantes del ABI, DESC_DIR(slot), includes
spl/include/layout_datos.inc	Zonas de datos, DESP_*, macros VEC_DIR y MAT_DIR
spl/include/vec.asm · cabeceras.asm · mat.asm	Primitivas 1D, cabeceras/descriptores, primitivas 2D
spl/estructuras/descriptores.py	DescriptorVector, DescriptorMatriz, MapaEstructurasRAM, normalización a 64 bits
spl/estructuras/memoria.py	Precarga y lectura de vectores, matrices y descriptores en RAMMemory
spl/estructuras/cargador.py	compilar_a_binario_e64, cargar_y_ejecutar_estructuras, .e64
spl/estructuras/algoritmos.py	Ejecutores de los tres problemas y constantes DESP_*
spl/ejemplos/*.asm	merge_sort, producto_punto, busqueda_binaria, multiplicacion_matrices y las pruebas prueba_vec, prueba_mat, prueba_cabeceras_cargador
enigma64/ui/servicios/estructuras.py	AdaptadorEstructuras: librería + cargador + RAM + CPU + MMIO
enigma64/ui/paneles/panel_estructuras.py · core/visores.py	Pestaña Tkinter, VisorVector y VisorMatriz

6.2 Requisitos y versiones

El emulador y la GUI usan solo la biblioteca estándar de Python (incluido tkinter). Las dependencias externas están en requirements.txt y environment.yml:

Componente	Versión probada	Requerida	Uso
Python	3.14.4	≥ 3.10	Lenguaje de implementación
PLY (ply.lex)	3.11	≥ 3.11	Preprocesador y analizadores léxicos del SPL
pytest	9.2.0	≥ 8.0	Suite de pruebas
Tk / tkinter	8.6	8.6	GUI Noctua (solo para la interfaz)

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt            # o: conda env create -f environment.yml
```

6.3 API de programación (Python)

```
from spl.estructuras import (ejecutar_merge_sort, ejecutar_producto_punto,
    ejecutar_busqueda_binaria, ejecutar_multiplicacion_matrices, cargar_y_ejecutar_estructuras,
    DescriptorVector, BASE_VECTORES)

ordenado, ej = ejecutar_merge_sort([45, -12, 89, 0])    # corre merge_sort.asm en la CPU
print(ordenado, ej.salida_mmio)                         # [-12, 0, 45, 89] '-12 0 45 89'
dot, _ = ejecutar_producto_punto([1, 2, 3], [4, 5, 6])  # 32
C, y, _ = ejecutar_multiplicacion_matrices([[1, 2], [3, 4]], [[5, 6], [7, 8]], vector_x=[1, 1])

# Programa propio con datos precargados:
v = DescriptorVector(nombre="V", elementos=[5, 1, 4], direccion_base=BASE_VECTORES, slot_cabecera=0)
ej = cargar_y_ejecutar_estructuras("mi_programa.asm", es_archivo=True, estructuras=[v])
```

6.4 Cómo extender la librería

- Para una primitiva nueva, añadirla al módulo que le corresponde (1D en vec.asm, 2D en mat.asm), respetar la ABI de la Sección 4.1 y no escribir nombres de macros en los comentarios (el preprocesador también expande dentro de los comentarios con ';').
- Para un módulo nuevo, crear el archivo en spl/include/ con #pragma once, guarda #ifndef y JMP inicial, e incluirlo desde estructuras.inc (opcionalmente bajo un #ifdef como INCLUIR_MATRICES).

- Para una tercera o cuarta estructura en un programa, usar las bases BASE_* + DESP_* de layout_datos.inc; la prueba test_layout_capacidad.py comprueba que Python y ensamblador coinciden.
- Un programa nuevo en spl/ejemplos/ que haga #include "estructuras.inc" aparece solo en el catálogo de la GUI; para que sus estructuras se dibujen solas basta registrarlas en OBSERVABLES_CONOCIDOS (o pulsar «Detectar en RAM»).

6.5 Pruebas

```
python -m pytest tests -q                                    # suite completa del proyecto
python -m pytest tests/test_libreria_estructuras.py tests/test_spl/test_vec_lib.py \
       tests/test_spl/test_mat_lib.py tests/ui/test_ui_estructuras.py -q
```

7. MANUAL DE USUARIO

7.1 Línea de comandos

```
# Preprocesar + ensamblar + empaquetar un programa que usa la libreria
python -m spl.estructuras spl/ejemplos/merge_sort.asm -o merge_sort.e64
# ... y ejecutarlo en el emulador (muestra registros, estructuras y salida MMIO)
python -m spl.estructuras spl/ejemplos/merge_sort.asm --ejecutar
# Solo el preprocesador, para ver la libreria expandida
python -m spl.preprocesador spl/ejemplos/producto_punto.asm
```

7.2 Interfaz gráfica: pestaña Estructuras

Se abre con `python -m enigma64.ui` (pestaña **Estructuras**, que comparte RAM, registros, CPU y MMIO con las demás pestañas) o, de forma aislada, con `python -m enigma64.ui.paneles.panel_estructuras`. La pantalla tiene tres zonas: a la izquierda los datos de entrada, arriba el programa y sus controles de ejecución, y en el centro los visores de vectores (celdas indexadas V[0..N−1]) y matrices (cuadrícula F × C).

1. **Elegir el programa** en el selector (merge sort encabeza la lista; luego producto punto, búsqueda binaria, multiplicación de matrices y las pruebas de la librería) o «Abrir .asm…». La descripción indica qué estructuras lee el programa, por ejemplo «A (vector en 0x00220000), B (vector en 0x00221000)».
2. **Escribir los datos** (opcional): tipo Vector o Matriz, nombre (A, B, V, Q, x…), base «auto» y valores (`45, -12, 89` para vectores; una fila por línea o separadas por «;» para matrices; se aceptan decimal, 0x…, 0b… y reales). «Escribir en RAM» la guarda con su cabecera y descriptor. Con base «auto», un nombre que el programa espera cae justo donde el programa lo lee; si se escribió antes de elegir el programa, se recoloca al elegirlo.
3. **Compilar y cargar**: preprocesa (resuelve el #include), ensambla, empaqueta el .e64, lo carga con CargadorEnigma en 0x00200000, pone SP = 0x00400000 y reescribe los datos de entrada.
4. **Ejecutar**: «Paso» avanza una instrucción e ilumina la celda leída (cian) o escrita (ámbar); «Animar» corre a la velocidad elegida y se puede pausar; «Hasta el final» corre hasta HLT. La barra muestra PC, la subrutina actual (p. ej. MERGE o MAT_GET), R1–R5, SP y BP.
5. **Leer resultados**: cada visor muestra el valor final y, debajo, el valor al cargar (entrada vs. salida); el formato puede ser decimal, hexadecimal o double. Pulsar una celda muestra su dirección (Dir = B + ((i·C + j) << 3)) y salta a ella en la pestaña Memoria. La salida de la pantalla MMIO aparece al pie.
6. **Detectar en RAM** recorre la tabla de descriptores y dibuja cualquier estructura que un programa propio haya creado con DESC_VEC_INIT / DESC_MAT_INIT.

Los errores (valores no numéricos, filas de distinta longitud, una base que se solapa con otra estructura, un programa que no ensambla) se muestran en rojo junto al control que los produjo y no detienen la interfaz.

8. EXPERIMENTACIÓN Y ANÁLISIS DE RESULTADOS

8.1 Metodología

Cada escenario se ejecuta sin intervención manual con el puente del Integrante 4: (1) el preprocesador resuelve `#include "estructuras.inc"`; (2) el ensamblador de la Tarea # 10 genera el código; (3) se empaqueta el .e64 y CargadorEnigma lo carga en 0x00200000; (4) se precargan los datos con sus descriptores; (5) la CPU multiciclo ejecuta hasta HLT; (6) se leen las estructuras de la RAM y se comparan con un oráculo en Python. Los datos son aleatorios con semilla fija (2026). Las cifras se obtuvieron el 11 de noviembre de 2026 con Python 3.14.4 y PLY 3.11; cada instrucción tarda 5 ciclos (FETCH, DECODE, EXECUTE, MEMORY, WRITE-BACK).

8.2 Escenario 1: algoritmo de las mezclas

Datos de demostración. Entrada V = [45, –12, 89, 0, 23, –5, 77, 10]. Salida en RAM [-12, -5, 0, 10, 23, 45, 77, 89] y en la pantalla MMIO «-12 -5 0 10 23 45 77 89». Resultado: correcto.

N	Instrucciones	Ciclos	Instr. / (N log2 N)	Resultado
8	2.082	10.410	86.8	ordenado ✓
16	5.217	26.085	81.5	ordenado ✓
32	11.959	59.795	74.7	ordenado ✓
64	27.122	135.610	70.6	ordenado ✓
128	59.953	299.765	66.9	ordenado ✓
256	131.974	659.870	64.4	ordenado ✓

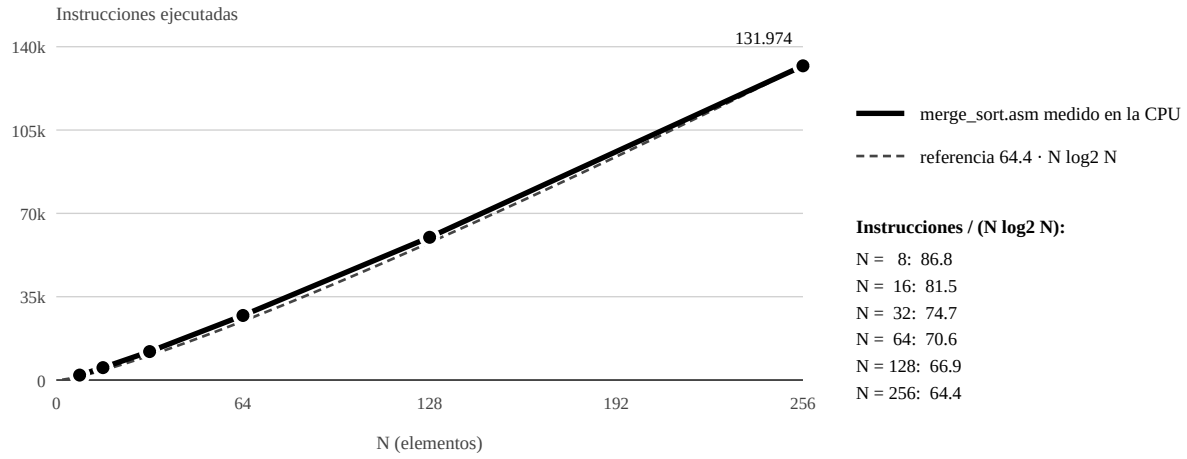


Diagrama 5: Instrucciones ejecutadas por merge_sort.asm frente a N, con la curva c·N log2 N ajustada al punto N = 256.

El cociente instrucciones / (N log2 N) baja de 86,8 a 64,4 y se estabiliza: el término lineal de la inicialización y de la salida MMIO pesa cada vez menos y el crecimiento observado es el Θ(N log N) que predice la teoría. Duplicar N de 128 a 256 multiplica las instrucciones por 2,20 (el modelo N log N predice 2,29). El costo constante por elemento y nivel (~64 instrucciones) proviene de que cada acceso pasa por VEC_GET / VEC_SET (CALL + 3 instrucciones + RET): es el precio explícito de la modularidad, que se cuantifica en 8.5.

8.3 Escenario 2: producto punto y búsqueda binaria

Demostración. A = [3, –4, 7, 2, –5], B = [6, 5, 2, –3, –4]: dot(A, B) = 26 (esperado 26). Búsqueda en V = [–20, –5, 0, 7, 15, 23, 42, 88] de Q = [23, –20, 88, 19]: R = [5, 0, 7, -1] (esperado [5, 0, 7, –1]).

N	Producto punto: instr.	Instr. / N	Búsqueda (4 consultas): instr.	Resultado
8	261	32.6	572	✓
32	837	26.2	756	✓
128	3.141	24.5	940	✓
256	6.213	24.3	1.032	✓

El producto punto crece linealmente (~24 instrucciones por elemento a partir de N = 32). La búsqueda binaria pasa de 572 a 1.032 instrucciones cuando N crece 32 veces: cada duplicación de N añade una iteración por consulta (~23 instrucciones × 4 consultas ≈ 92, exactamente la diferencia entre 128 y 256), la firma de Θ(log N).

8.4 Escenario 3: multiplicación de matrices y matriz por vector

Demostración. A = [[1, 2, 3], [4, –1, 2]], B = [[2, 1], [0, –3], [4, 5]], x = [2, –1, 3]: C = [[14, 10], [16, 17]] y y = [9, 15], ambos iguales al cálculo a mano.

n (n × n)	C = A·B: instr.	Instr. / n^3	y = A·x: instr.	Resultado
2	642	80.2	—	✓
4	3.374	52.7	727	✓
8	23.430	45.8	2.399	✓
12	76.062	44.0	—	✓
16	177.014	43.2	8.911	✓

El cociente instrucciones / n^3 tiende a ~43, lo que confirma el costo Θ(n^3) del producto (de 8 a 16 el trabajo se multiplica por 7,6 ≈ 2^3). El producto matriz por vector crece como n^2 (de 8 a 16: × 3,7). Con el mapa original (B a 512 bytes de A) un producto 8 × 8 era rechazado por solapamiento; con DESP_MATRIZ_2 = 0x3000 caben matrices de 32 × 32, el máximo que dibuja la GUI.

8.5 Suite de regresión

Archivo de pruebas	Casos	Qué verifica
test_preprocesador_estructuras.py	31	#include, guardas, macros VEC_DIR / MAT_DIR, layout
test_vec_lib.py	41	VEC_* en la CPU real: direcciones, solapes de VEC_COPY, preservación
test_mat_lib.py	74	MAT_*, contenido de cada marco de pila, recursión de 300 niveles
test_estructuras_cargador.py	18	descriptores, normalización 64 bits, .e64 y CargadorEnigma
test_merge_sort.py	20	merge sort: casos borde, estrés de recursión, métricas N log N
test_libreria_estructuras.py	38	los tres problemas extremo a extremo
test_layout_capacidad.py	8	DESP_* iguales en Python y ensamblador; vectores de 300, 8 × 8
tests/ui/test_ui_estructuras.py	43	servicio y pestaña de la GUI con los tres problemas
Total Tarea 29	273	

La suite completa del proyecto (incluidas las Tareas # 10, # 17, # 27 y el Taller # 01) se ejecutó tras integrar las ramas de los siete integrantes: **1.786 pruebas pasan y 1 se omite (además de 419 subpruebas)**, lo que confirma que la librería no alteró la máquina ni las herramientas previas.

8.6 Análisis de resultados

Corrección. En los tres escenarios los resultados coinciden con el oráculo para todos los tamaños probados, incluidos negativos, duplicados, vectores de un elemento y consultas ausentes. **Costo de la modularidad.** En el merge sort una lectura de V[i] cuesta 5 instrucciones vía VEC_GET (CALL, SHL, ADD, LOAD, RET) frente a 3 en línea; en el producto de matrices MAT_GET añade además el marco ENTER/LEAVE y la llamada anidada a MAT_ADDR (12 instrucciones por acceso, contando la llamada). Ese factor constante (≈ 1,7–3×) no cambia el orden de complejidad, como muestran las tablas, y compra legibilidad, reutilización y que cada rutina se pruebe de forma aislada. Las macros VEC_DIR / MAT_DIR quedan disponibles para el programador que quiera el acceso en línea cuando el índice es constante. **Integración.** La revisión de la integración detectó que los programas 2 y 3 colocaban la segunda estructura a 256 / 512 bytes de la primera: un vector de 31 elementos o una matriz 8 × 8 producían un solapamiento (detectado por MapaEstructurasRAM, nunca corrupción silenciosa). Se centralizaron los desplazamientos en layout_datos.inc y se añadió la prueba de consistencia; la GUI toma las mismas constantes.

9. LIMITACIONES CONOCIDAS

- Las primitivas no comprueban límites (como los arreglos de C): un índice fuera de rango lee o escribe memoria vecina. La validación se hace en el lado Python y en la GUI antes de escribir.
- Los elementos son palabras de 64 bits; la librería mueve bits crudos y el cálculo con double depende del programa (la FPU de la Tarea # 17), no de la librería.
- La capacidad por estructura la fija el mapa: 510 elementos para el primer vector de un programa con dos vectores y 1.534 palabras para cada matriz; los programas de prueba usan bases fijas.
- Las estructuras son estáticas: no hay asignación dinámica (malloc) ni redimensionamiento.
- Un preprocesador de una línea por macro no puede emitir secuencias de instrucciones, por eso la gestión de pila vive en subrutinas y no en macros.

10. CONCLUSIONES

- Se construyó una librería de listas y matrices para Enigma-64 que se incorpora con una sola línea (`#include "estructuras.inc"`) y que gestiona el preprocesador del SPL, como pide el enunciado, sin modificar la máquina de la Tarea # 10.
- El diseño modular (layout, vec, cabeceras, mat; guardas de inclusión; ABI única) permitió que siete personas desarrollaran y probaran su parte por separado y que todo se integrara sin conflictos.
- El algoritmo de las mezclas, el producto punto, la búsqueda binaria y el producto de matrices corren en la CPU emulada con resultados correctos y con los órdenes de crecimiento predichos: $\Theta(N \log N)$, $\Theta(N)$, $\Theta(\log N)$ y $\Theta(n^3)$.
- La pestaña Estructuras de la GUI hace visible el funcionamiento: los datos que escribe el usuario, las celdas que cada instrucción lee o escribe y el resultado final frente a la entrada.
- 273 pruebas automatizadas específicas de la tarea, sobre la CPU real, sostienen la corrección y protegen el contrato de la librería ante cambios futuros.

REFERENCIAS

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022.

[2] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compiladores: principios, técnicas y herramientas*, 2.ª ed. Pearson, 2008.

[3] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 6.ª ed. Morgan Kaufmann, 2020.

[4] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2.ª ed. Addison-Wesley, 1998.

[5] D. Beazley, *PLY (Python Lex-Yacc) 3.11*, documentación, https://www.dabeaz.com/ply/.

[6] Grupo Enigma-64, *Tareas # 09, # 10, # 17, # 27 y Taller # 01*, Universidad Nacional de Colombia, 2026.

A. ANEXO: INVENTARIO DEL CÓDIGO FUENTE

Archivo	Líneas	Autor
spl/include/estructuras.inc	67	Integrante 4
spl/include/layout_datos.inc	84	Integrantes 1 y 5
spl/include/vec.asm	151	Integrante 2
spl/include/cabeceras.asm	155	Integrante 4
spl/include/mat.asm	139	Integrante 3
spl/estructuras/ (descriptores, memoria, cargador, __main__, __init__)	1.344	Integrante 4
spl/estructuras/algoritmos.py	267	Integrante 7
spl/ejemplos/merge_sort.asm	362	Integrante 6
spl/ejemplos/producto_punto.asm · busqueda_binaria.asm	414	Integrante 7
spl/ejemplos/multiplicacion_matrices.asm	419	Integrante 7
spl/ejemplos/prueba_vec.asm · prueba_mat.asm · prueba_cabeceras_cargador.asm	356	Integrantes 2, 3 y 4
enigma64/ui/servicios/estructuras.py	896	Integrante 5
enigma64/ui/paneles/panel_estructuras.py · core/visores.py	931	Integrante 5
tests/ (8 archivos de la Tarea 29)	2.933	Todos
docs/integrantes/informe_integrante*_{vec,mat,estructuras_*,merge_sort_mmio}.md	—	Informes individuales